Society of Petroleum Engineers

SPE-229491-MS

Automated Multi-Well Multi-Property Calibration of Basin and Petroleum System Models in a Massively Parallel High-Performance Computer Environment

M. A. Al Ibrahim, EXPEC Advanced Research Center, Saudi Aramco; R. Ahmed and G. Port, Strategic Modeling Technology, Saudi Aramco; N. Alhijri, EXPEC Advanced Research Center, Saudi Aramco

 DOI 10.2118/229491-MS

This paper was prepared for presentation at the ADIPEC held in Abu Dhabi, UAE, 3 – 6 November 2025.

Abstract

Accurate basin and petroleum system models are crucial for understanding the complexities of petroleum systems, including predicting potential hydrocarbon accumulations, their timing, and migrating oil and gas phase behavior. These models rely on calibration with measured data including subsurface temperatures and thermal maturity indices. Calibrating this measured data is essential because the transformation of kerogen into hydrocarbons is driven by temperature-dependent kinetic reactions. Traditional calibration methods are often manual and time-consuming, but they are still used because of the high computational costs of optimization algorithms and/or limitations in existing simulators. This paper demonstrates that by leveraging a purpose-built simulator for high-performance computing environments and state of the art optimization algorithms, automated calibration of these models is not only feasible but also highly effective. Three components enable practical automated calibration of these models: 1) a simulator specifically designed to be run in a massively parallel high-performance computing environment is used, 2) an easy-to-use Python interface to parse and write input files and read output files for the simulator, and 3) the vast scientific Python libraries. To automatically calibrate basin models, an initial (uncalibrated) model is constructed. Uncertain parameters are defined either as numerical range, discrete values, or possible scenarios. Bayesian optimization with sampler based on Tree-structured Parzen Estimator is used for calibration. The automation workflow is applied on multiple real three-dimensional models to test the viability of the method. In one model, the uncertainty in basal heat flow and surface temperature histories is represented as a bulk shift in the initial maps used (±30%). Calibration data used are the borehole measured temperature and vitrinite reflectance with the first being weighted twice as the latter during error calculation. Results show that the optimization algorithm is able to calibrate the four wells at the same time. The optimization algorithm samples more realizations/solutions closer to the local minima in the response surface. This is because the optimization algorithm attempts to balance between exploitation (generating realizations near local minima) and exploration (generating realizations in areas with sparse response). In addition, in this case, the model is more sensitive to the basal heat flow history rather than the surface temperature history. Improvements that can be implemented to accelerate the optimization includes utilizing parallel simulations and halting the

optimization once a predefined calibration threshold is achieved. Overall, the developed framework opens opportunities to apply different state of the art algorithms to aid in basin and petroleum system modeling. The novelty in the work lies in the fact that by combining multiple technologies, a practical workflow for automated calibration of realistic basin and petroleum system modeling is achievable. Thus, previous limitations, such as computational constraints, on such workflows is overcome.

Introduction

Basin and petroleum system simulation is a crucial step in understanding the behavior of complex geological systems and predicting the potential for hydrocarbon accumulation (Hantschel and Kauerauf, 2009). These simulations involve the use of sophisticated models that account for various geological processes, such as sedimentation, compaction, and fluid flow. The construction of these models involves quantitively estimating a large number of parameters based on available subsurface data in the area and possible analogues, both modern and ancient. The output of these simulation ultimately contributes to the operational decisions, and so, it is important to obtain accurate results and understand the associated uncertainty (Figure 1). The accuracy of these models relies heavily on their calibration, which is the process of adjusting the model parameters to match the observed data. Calibration of these models is a time-consuming process when done manually, especially when dealing with complex models that involve multiple wells, variable data types, and computationally intensive simulations (e.g., when simulating three-dimensional models). The manual calibration process requires significant expertise and can be prone to errors, making it a challenging task even for experienced professionals. The complexity of the model, the abundance and variability of calibration data across multiple wells, and the uncertainty in the input parameters all contribute to manual calibration difficulty.

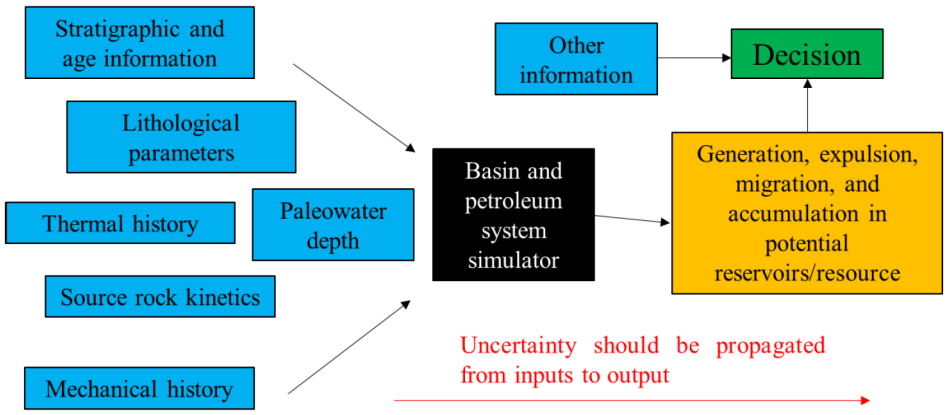


Figure 1—A schematic diagram of general workflow for basin and petroleum system modeling. Large number of parameters are inputted into the simulation. The simulator produces results which focuses on hydrocarbon accumulations. This information plays a vital role in hydrocarbon exploration decisions.

To overcome these challenges, automated calibration techniques, such as optimization algorithms and machine learning methods, can be used to efficiently search the parameter space and identify the optimal solution. Previous studies have showed that this is possible. Hantchel and Kauerauf (2009) used Markov Chain Montel Carlo (MCMC) for calibration of small models. Hosford Scheirer (2019) estimated the posterior using MCMC and found the best fit model from the results. Al Ibrahim (2023) automated the calibration of one-dimensional models. None of these tackled the use case where the model is complex and three-dimensional, calibration data is abundant and of variable types, and/or uncertainty in the input is relatively large. These issues make automated calibration practically unfeasible. This paper tackles these issues. The study will show that by combining a highly parallel basin and petroleum simulator with state-of-the-art optimizers, it is possible to implement a practical solution for automated calibration of realistic

models. The paper will discuss the methods and workflow in some details, followed by examples covering multiple cases, and a discussion of results and possible improvements.

Workflow and Methods

Three components enable practical automated calibration of basin and petroleum system models: 1) a simulator specifically designed to be run in a massively parallel high-performance computing environment, 2) an easy-to-use Python library to interface with the simulator and read/write input files and read output files, and 3) Python scientific optimization and machine learning libraries. The workflow is implemented in Python using open source libraries (McKinney, 2010; Akiba et al., 2019; Harris et al., 2020). Figure 2 shows a typical high-level code developed. In addition, A user interface is designed to simplify operations for zero-code users. The following section will discuss the details of the implementation.

```python
present_heatflow_range = [5, 55]
property_values_tested = []
cost_values_tested = []
wells_result_tested = []

def objective(trial:optuna.Trial):
    """Objective function"""

    # Suggest the property
    value = trial.suggest_float('present_heat_flow', present_heatflow_range[0], present_heatflow_range[1])
    property_values_tested.append(value)

    # Update the property
    for i in range(3):
        heat_flow_map = basal_heat_flow.strata[i].maps[0]
        update_map_with_uniform_value(heat_flow_map, value)

    # Run the model
    job_id = submit_simulation(submission_file_filename, filesystem)
    # print(f"Job id: {job_id} submitted with parameter {value}")

    # Wait until simulation is finished
    while not is_job_completed(job_id, filesystem):
        time.sleep(5)

    # Get the results
    simulated_data, wells_result = get_well_simulated_data(wells_results_filename, property_to_calibrate, filesystem)
    wells_result_tested.append(wells_result)

    # Calculate the loss
    cost = mean_squared_error(simulated_data, calibration_data)
    cost_values_tested.append(cost)
    return cost

sampler = optuna.samplers.TPESampler()
study = optuna.create_study(study_name="TeraBasin Optimization", sampler=sampler)
study.optimize(objective, n_trials=50)

present_heat_flow_calibration = (present_heatflow_range, property_values_tested, cost_values_tested, wells_result_tested)

# Save things
with open("present_heat_flow_calibration.pkl", "wb") as f:
    pickle.dump(present_heat_flow_calibration, f)
joblib.dump(study, "present_heat_flow_calibration.study")
```

Figure 2—Example code used in the workflow. This example uses Optuna's Bayesian optimization framework.

Basin and Petroleum System Modeling Simulator

A highly parallel custom-built basin and petroleum system simulator is used in this study (Dogru et al., 2018; Ahmed et al., 2025). The basin simulator is developed on top of a well-tested framework for parallel reservoir simulation (Dogru et al., 2009). The simulator runs on a high-performance computing environment and is capable of simulating models with billions of cells with Darcy flow with over half a billion years of geologic time in a few hours (Dogru et al., 2018). This is because the calculation is distributed over multiple cores in an efficient manner. In essence, the simulator follows the steps as outlined: 1) depositional analysis which includes sedimentation and decompaction and calculation of rock and fluid properties as needed, 2) pressure calculation, 3) heat flow calculation, 4) hydrocarbon generation/adsorption and expulsion, 5) hydrocarbon migration, and 6) reservoir volumetrics analysis. Ahmed et al. (2025) provides more detailed if needed. To facilitate the interface between the simulator and advanced statistical algorithms implemented by the Python scientific community in the last few decades, A Python library was developed. The library was designed to: 1) read and write the simulator input, 2) control the simulator including job submission and checking on simulation status, and 3) read the simulation output.

Automated calibration

The automated calibration is performed using optimization algorithms. in the context of automated calibration, a number of terms must be defined: uncertain parameters, calibration data, objective function, and well and calibration data type weights. The following section describe these in some detail with focus on their meaning in basin and petroleum system modeling.

Uncertain input parameters are model parameters that will be modified automatically to reach the optimum calibration. Typical uncertain parameters modified include: 1) thermal history including surface temperature and basal heat flow, 2) porosity-permeability relationships, and 3) source rock characteristics such as total organic content and hydrogen index. The choice of the uncertainty ranges (and possibly the distribution) is based on the user expertise and available data. Note that some algorithms, such as the Optuna implementation of the TPE Bayesian optimization which will be discussed below, does not require the explicit definition of the distributions. This can be an advantage the user only needs to define the minimum and maximum values. In general, the uncertain parameters can be: 1) continuous floating values, 2) integers, and/or 3) scenarios of certain properties. For example, multiple erosion maps for a certain hiatus can be designed an incorporated into the optimization. It is possible to optimize all different types simultaneously.

Calibration data is data available from external sources that can be used to validate the model simulated. The data can include: 1) present day borehole temperature measurement, 2) present day vitrinite reflectance measurements, 3) borehole pressure measurements, 4) borehole mud weight, and 5) estimated accumulation height. Any data that is available from an external measurement and is part of the output of the simulation can be used. Note that each data type has its own uncertainty that can be incorporated into the optimization during the design of the objective function.

The objective function defines the comparison between the simulation results to some measured or estimated values. The optimization algorithm attempts to minimize this value by adjusting the uncertain input parameters. The values typically compared include borehole temperature at certain depth, vitrinite reflectance at certain depth, and/or accumulation column height at certain locations. Typical method to calculate this difference is to use the Euclidean distance metric. Equation 1 defines for Euclidean distance ($D_{Euclidean}$) where N is the number of calibration data points, and v_k is the kth data point. Other metrics such as the absolute distance can also be used; if there is a justifiable reason. For example, a distance metric can be designed to penalize an overestimation in a higher manner than an over estimation. This might be useful when comparing simulated pore pressure values to mud weights used in the well; while the two values are not equivalent, mud weights can be considered as an upper limit to pore pressure.

$$D_{\text{Euclidean}} = \frac{1}{N}\sqrt{\sum_{k=1}^{N}\left(v_{k,\ simulated} - v_{k,\ measured}\right)^2} \tag{1}$$

Calibration data type weights defines the importance of each data type used in the calibration. When calibrating based on multiple properties at the same time, e.g., present day temperature and vitrinite reflectance, it is important to consider the range of each property. In the case of the example given, present day temperature will have a much higher values compared to vitrinite reflectance. In this case, the calculation of the objective function would overweigh the present-day temperature. To resolve this, each property should be normalized. Typical methods for normalization are z-score and min-max-normalization. Z-Score normalization uses the mean (*mean*) and standard deviation (*std*) of the data (Equation 2) where min-max normalization uses and minimum (*min*) and maximum (*max*) values (Equation 2). Once normalization is performed, it is possible to increase the importance of a certain data type intentionally by using a multiplier for the data type. Thus, Equation 1 is extended to Equation 4 when multiple properties are used as calibration data where $w_{v\text{-}type}$ defines the weight of the calibration data type. An example of usage is to define present day temperature calibration data points to have a higher weight (say 2) compared to vitrinite reflectance because the uncertainty in the temperature measurements is lower.

$$v_{k,norm_{z-score}} = \frac{v_k - \text{mean}(v,\quad measured)}{\text{std}(v,\quad measured)} \tag{2}$$

$$v_{k,\ norm_{min-max}} = \frac{v_k - \min(v,\quad measured)}{\max(v,\quad measured) - \min(v,\quad measured)} \tag{3}$$

$$D_{\text{Euclidean,weighted}} = \frac{1}{N}\sqrt{\sum_{k=1}^{N} w_{v_k type}\left(v_{k,\ norm,simulated} - v_{k,\ norm,measured}\right)^2} \quad (4)$$

Note that if multiple wells are used in the calibration, two approach can be taken. The first is to weigh the wells based on the number of calibration data points available. A second approach is to weigh each well using a user-defined multiplier. The first approach is what is defined in Equation 4. The second approach requires an extra modification to normalize each well by the number of calibration data points in it and using a multiplier (in a similar fashion to the approach taken for data type normalization above). The choice of the approach depends on the use case. Generally, the first approach is taken unless there is a justifiable reason to take the second approach. An example of a justification is the existence of a calibration well in an important isolated area of the model far away from other calibration wells. In this case, the user might increase its importance to ensure a good calibration in that area.

Optimization algorithms

The study showcases the use of two optimization algorithms: Bayesian optimization using Tree-structured Parzen Estimator, and genetic algorithm. The choice between optimization algorithms depends on the available resources and the complexity of the use case. For example, genetic algorithms might be preferred when there are enough resources to run large number of simulations in parallel as the core algorithm is designed to handle this case.

Bayesian optimization is used for the automated calibration as implemented in Optuna (Akiba et al., 2019). The algorithm uses a Tree-structured Parzen Estimator (TPE) to efficiently search for the optimal parameters (Bergstra et al., 2011). The TPE algorithm works by modeling the objective function as a probability distribution over the search space, where the probability of a parameter being optimal is estimated based on its past performance. As the optimization process progresses, the algorithm adaptively updates the probability distribution, focusing on the most promising regions of the search space and pruning less promising areas. This approach effectively balances exploration and exploitation, leading to rapid convergence to the optimal solution. By leveraging the TPE algorithm, can efficiently optimize complex objective functions with multiple local optima, making it a powerful tool for machine learning hyperparameter tuning and other general optimization tasks where the response surface is complex.

Genetic algorithm is an optimization technique used to solve both constrained and unconstrained problems by searching for local or global minima (Alhijawi and Awajan, 2024). It is inspired by the concept of biological evolution. The process begins by generating an initial population composed of multiple individuals, where each individual represents a potential solution to the problem at hand. In this work, we applied genetic algorithm to optimize the temperature variable in order to calibrate the well simulation data of the model. Each individual in the population is evaluated using a fitness function, which analyzes the second norm of the distance between the well simulation and calibration data to assess how well the individual solves the problem. Based on these fitness scores, a subset of individuals, referred to as parents, is selected to produce children for the next generation. This selection process is typically stochastic, meaning that individuals with higher fitness scores have a greater probability of being chosen. The evolution of the population toward better solutions is driven by three key genetic operations: selection, crossover, and mutation. During selection, individuals with higher fitness are chosen to serve as parents. In the crossover phase, pairs of parents are combined to generate new children by combining segments of their genetic information. Finally, mutation introduces minor and random changes to some individuals to maintain genetic diversity and help the algorithm avoid being stuck in local optima. Through repeated iterations of these steps, the population gradually evolves toward an optimal or near-optimal solution.

Examples

The first example shows automated calibration for finding the optimum basal heat flow multiplier and surface temperature multiplier. The three-dimensional model constructed with two million cells composing thirty-three stratigraphic layers. For wells used in the calibration with both temperature and vitrinite reflectance data with temperature being weighted twice as vitrinite reflectance during optimization. The model simulates in about thirteen minutes using the in-house simulator with 128 CPU cores. This shows the scalability of the simulator to enable statistical analysis and optimization workflows.

Uncertain parameters are defined as multipliers for both basal heat flow and surface temperature from the initial guess. The multiplier for each map type is applied in bulk on each map, i.e., the maps are shifted higher or lower in values. TPE-based Bayesian optimization is used here for automated calibration. Figure 3 shows that the optimization is able to find a good fit (Figure 3, well calibration plots). The response surface (Figure 3, bottom right) shows the values of the objective function as a function of the uncertain parameters. Note that the sampler in TPE-based Bayesian optimization selected more realizations closer to the minimum/optimum fit. This is in line with the algorithm where it attempts to balance between exploration and exploitation of the objective function space.

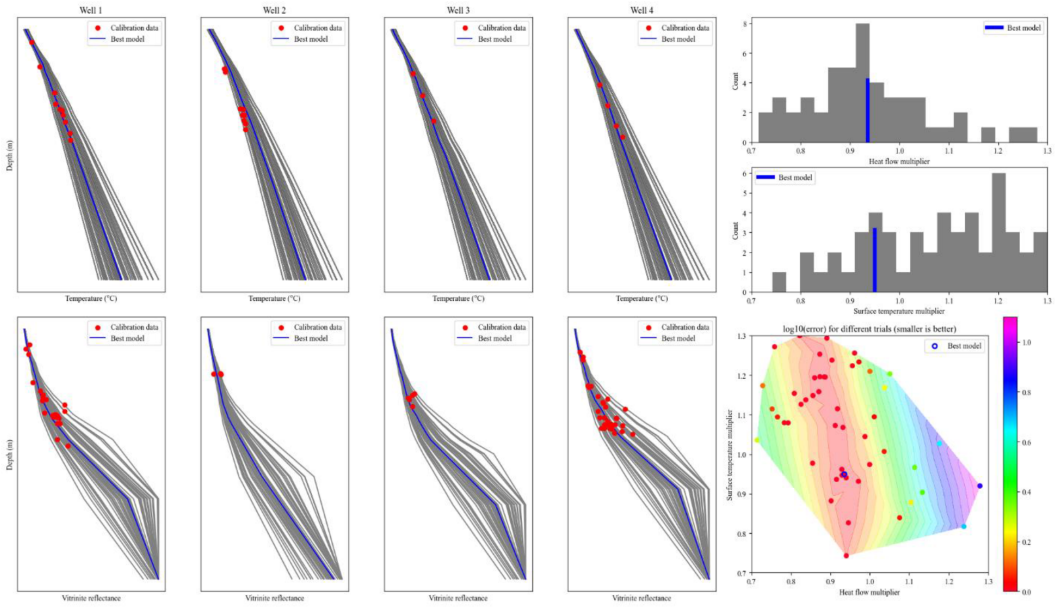
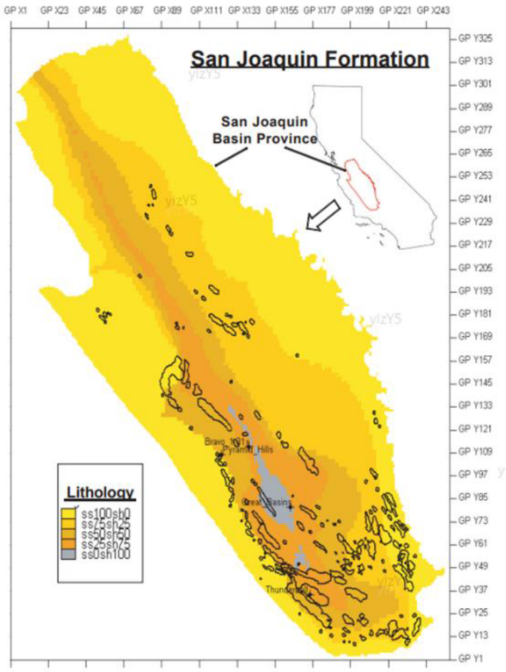


Figure 3—Results for optimization for the first example. Four wells are used for calibration. Basal heat flow and surface temperature multipliers are used as uncertain parameters. 50 realizations are simulated.

The results also shows that the objective function, which relates to the goodness of fit of the simulation to the calibration model, is more sensitive to the basal heat flow multiplier. The surface temperature is insensitive as changing the multiplier would not produce drastically different objective function values. This observation is quantified using fANOVA (Hutter et al., 2014), which calculates the parameter importance. The normalized values for the importance are 0.994 and 0.005 for basal heat flow and surface temperature respectively. This shows that surface temperature is not important and, in fact, can virtually be ignored during the optimization. This information might also be valuable for subsequent studies as the expert can put larger effort in constructing basal heat flow maps compared to surface temperature maps. Note that other algorithms for parameter importance can also be used, e.g., SHAP (Lundberg and Lee, 2017).

For the second example, the San Joaquin basin model is used. Figure 4 shows the location, general lithology distribution of the San Joaquin formation and the basin model stratigraphy used. The three-dimensional model is adapted from Peters et al. (2008). The model is composed of 24 stratigraphic units,

spans 249 × 209 km area, and is composed of 1.8 million grid cells. Multiple reservoirs and source rocks exist in the stratigraphic section. Calibration data available are present day temperature five wells. The model simulates in about three minutes using the in-house simulator with 128 CPU cores.



Age of Interval (Era/Epoch)			Stratigraphic Unit	Deposition (Ma)		Hiatus (Ma)		Petroleum System Element
				From	To	From	To	
Cenozoic	Tertiary	Pliocene to Pleistocene	Tulare Formation	3.5	0.0			Overburden-reservoir rock
		Pliocene	San Joaquin Formation	4.8	3.5			
		Miocene to Pliocene	Etchegoin Formation	6.0	4.8			
		Miocene	*Antelope shale*, nonsource	11.0	6.0			
			Antelope shale (top)	15.5	11.0			Source rock
			Stevens sand	15.5	15.5			Reservoir rock
			Antelope shale (bottom)	16.5	15.5			Source rock
			Temblor Formation	18.5	16.5			Reservoir rock
		Oligocene to Miocene	Temblor Formation	36.0	18.5			Seal rock
		Eocene to Oligocene	*Tumey formation*	38.0	36.0			Source rock(s)
		Eocene	Kreyenhagen Formation	42.0	38.0			
			Point of Rocks Sandstone Member of Kreyenhagen Formation	43.0	42.0			Reservoir rock
			Kreyenhagen Formation	48.0	43.0			Source rock
		Paleocene to Eocene	Domengine Formation	61.0	48.0			Reservoir rock
		Paleocene	Garzas Sandstone Member of Moreno Formation	64.0	61.0			Reservoir rock
Mesozoic	Cretaceous to Paleocene		Moreno Formation	72.0	64.0			Source rock
	Cretaceous		*Ragged Valley silt*	73.0	72.0			Reservoir rock
			Tracy sands	73.5	73.0			Underburden
			Sawtooth shale	75.0	73.5			
			Lathrop sand	77.0	75.0			
			Sacramento shale	78.0	77.0			
			Forbes formation	85.0	78.0			
	Cretaceous and older		weathered basement	106.0	105.0	105.0	85.0	Basement
			fresh basement	162.0	106.0			

Figure 4—San Joaquin Formation (left) and basin model stratigraphy for the area (right). Figure is modified from Peters et al. (2008), a public domain USGS report.

The genetic algorithm demonstrates rapid convergence toward the optimal solution. As shown in Figure 5, after generating a small initial population of three individuals, the algorithm reached a mean squared error (MSE) of approximately 0.83 and a basal heat flow value of 20.44, very close to the optimal solution, within only 25 trials. During each population evaluation, the algorithm runs the simulator and computes the MSE for each individual to assess their performance. The results clearly show that convergence toward the best solution occurred quickly. This efficiency is primarily due to the selection mechanism, where parents are chosen based on their fitness scores. As a result, individuals with better performance have a higher chance of contributing to the next generation, effectively guiding the search process in the direction of improved solutions.

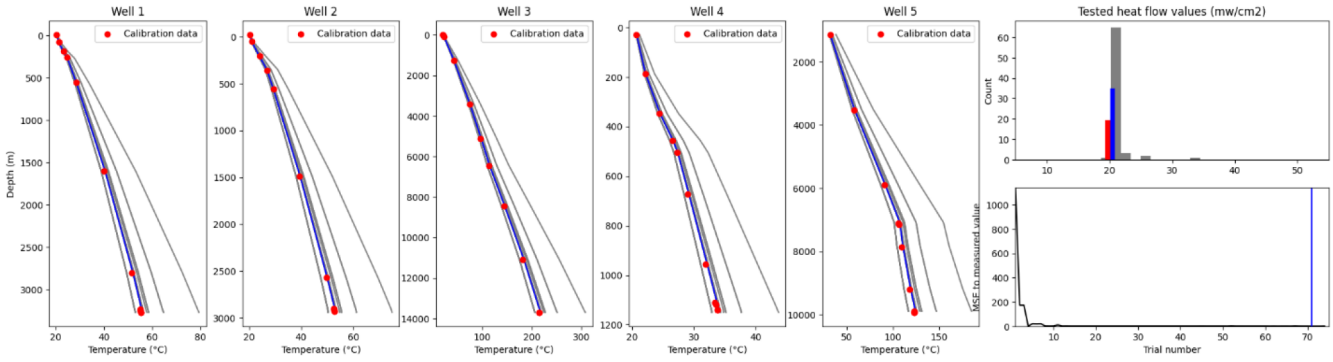


Figure 5—Temperature optimization using genetic algorithm for precise calibration of well simulation data in model.

Discussion

To improve the performance of the automated calibration workflow, a number of strategies can be implemented. If the main goal is to find a good calibrated model without interest in the uncertainty associated with it or sensitivity analysis, early stopping can be implemented. The details of the implementation are dependent on the optimization algorithm used. For example, if Bayesian optimization is used, a threshold for the objective function and the algorithm is stopped once an objective function value is reached that is lower. If genetic algorithms are used, a threshold can also be used. Alternatively, the results from one generation to the next can be compared and the optimization is stopped if no significant changes are observed. Another strategy to improve the performance of the Some optimization algorithms allow for parallel evaluation of the objective function. This means that multiple simulations can be run and evaluated. That being said, the maximum number of simulation run should be enough to allow multiple batches of models to run. This will allow the optimization algorithm to propose new realizations based on previous results which is important. Finally, for certain calibration where hydrocarbon migration is not needed, it can be disabled. For calibration of properties that relies on fluid flow such as column height, migration must be enabled.

Conclusions and final remarks

The combination of a highly parallel basin model simulator and advanced statistical algorithms opened the opportunity for practical automated workflows. This paper showed that it is now feasible to conduct automated calibration of models using complex three-dimensional models in an efficient manner. The use of an in-house developed simulator opens opportunities for tighter coupling with machine learning algorithms. The use of open-source tools for statistical analysis and optimization algorithms accelerates development. Overall, it is now possible to obtain more accurate basin and petroleum system models to assist in energy resource exploration and production.

References

Ahmed, A., G. Port, and R. Schmidt, 2025, A massively parallel basin simulation with multiphase hydrocarbon migration of a large-scale model, in Manama, Bahrain: SPE, p. SPE-227199-MS.

Akiba, T., S. Sano, T. Yanase, T. Ohta, and M. Koyama, 2019, Optuna: A next-generation hyperparameter optimization framework, in Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, New York, NY, USA: Association for Computing Machinery, KDD '19, p. 2623–2631, doi:10.1145/3292500.3330701.

Al Ibrahim, M. A., 2023, An open-source toolbox to automate basin and petroleum system modeling for statistical analysis and uncertainty quantification, in Houston, Texas, USA: p. SEG-2023-3914751, doi:10.1190/image2023-3914751.1.

Alhijawi, B., and A. Awajan, 2024, Genetic algorithms: theory, genetic operators, solutions, and applications: Evolutionary Intelligence, v. **17**, no. 3, p. 1245–1256, doi:10.1007/s12065-023-00822-6.

Bergstra, J., R. Bardenet, Y. Bengio, and B. Kégl, 2011, Algorithms for hyper-parameter optimization, in Proceedings of the 25th International Conference on Neural Information Processing Systems, Red Hook, NY, USA: Curran Associates Inc., NIPS'11, p. 2546–2554.

Dogru, A. H. et al, 2018, A highly parallel billion cell basin simulator, in Manama, Bahrain: AAPG, p. 90319.

Dogru, A. H. et al, 2009, A Next-Generation Parallel Reservoir Simulator for Giant Reservoirs: p. SPE-119272-MS, doi:10.2118/119272-MS.

Hantschel, T., and A. I. Kauerauf, 2009, Fundamentals of basin and petroleum systems modeling: Springer Berlin, Heidelberg, 476 p.

Harris, C. R. et al, 2020, Array programming with NumPy: *Nature*, v. **585**, no. 7825, p. 357–362, doi:10.1038/s41586-020-2649-2.

Hosford Scheirer, A., 2019, The value of integrating machine learning for enhanced understanding of petroleum systems, in Houston, Texas, USA: AAPG.

Hutter, F., H. Hoos, and K. Leyton-Brown, 2014, An efficient approach for assessing hyperparameter importance, in E. P. Xing, and T. Jebara, eds., Proceedings of the 31st International Conference on Machine Learning, Bejing, China: PMLR, Proceedings of Machine Learning Research, p. 754–762.

Lundberg, S. M., and S.-I. Lee, 2017, A unified approach to interpreting model predictions, in I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, eds., *Advances in Neural Information Processing Systems*, Long Beach, CA, USA: Curran Associates, Inc.

McKinney, W., 2010, Data structures for statistical computing in Python, in S. van der Walt, and J. Millman, eds., Proceedings of the 9th Python in Science Conference, Austin, Texas, USA: Scipy.org, p. 56–61, doi:10.25080/Majora-92bf1922-00a.

Peters, K. E., L. Magoon, C. Lampe, Allegra Hosford Scheirer, P. Lillis, and D. Gautier, 2008, A four-dimensional petroleum systems model for the San Joaquin Basin Province, *California: Chapter 12 in Petroleum systems and geologic assessment of oil and gas in the San Joaquin Basin Province*, California, pp171312: USGS, U.S. Geological Survey Professional Paper.